

Computer Organization & Design Project

16-Bit Basic CPU Design

TABLE OF CONTENTS

PG. NO

ACKNOWLEDGEMENT	1
1. INTRODUCTION.....	3
2. OBJECTIVES.....	4
3. ASSUMPTIONS AND INSTRUMENTS REQUIRED	5
4. ARCHITECTURE	6
5. CIRCUIT SCREENSHOTS AND EXPLANATION.....	9
5.1 REGISTERS.....	9
5.2 ARITHMETIC AND LOGIC UNIT	13
5.3 COMMON BUS	15
5.4 CONTROL UNIT	17
6. CONCLUSION.....	22
7. REFERENCES.....	23

INTRODUCTION

Project Overview: Logisim CPU Design Based on Morris Mano Architecture

Computer architecture is the foundation of modern digital systems, with the **CPU at its core**—responsible for fetching, decoding, executing instructions, and storing results. Understanding this process is essential for anyone working at the hardware-software interface.

This project is based on the **Morris Mano CPU model** from *Computer System Architecture* by M. Morris Mano, a widely respected educational framework. It uses a **hardwired control unit** with combinational and sequential logic, offering a clear view of CPU operations without the added complexity of microprogramming.

We designed a **16-bit CPU in Logisim Evolution**, closely following the Morris Mano architecture. The CPU includes core components like **registers, ALU, memory unit, control unit, and a timing system**. Each instruction is executed through precise micro-operations controlled by a sequence counter and clock, simulating the **T0 to T15 timing steps** defined in the model.

The CPU supports a **basic instruction set** for memory and register operations, branching, and simple logic. Advanced features like interrupts and pipelining are intentionally excluded to maintain focus on core CPU operations and improve conceptual clarity.

This project provides hands-on experience in **sequential logic, memory operations, buses, multiplexing, tri-state logic, and control signal propagation**—bridging the gap between theoretical knowledge and practical circuit design.

OBJECTIVES

- To understand and implement the instruction execution flow as proposed in Morris Mano's architecture, including the precise sequence of micro-operations required for fetching, decoding, and executing each instruction in a controlled, clock-driven manner.
- To simulate a 16-bit CPU with accurate micro-operations, timing signals, and register behaviors, replicating the internal processor activity such as data transfers, ALU operations, and memory access in a way that reflects the theoretical design of the Morris Mano model.
- To design a hardwired control unit for decoding instructions and issuing control signals, using combinational logic circuits, decoders, and encoders to translate opcodes and timing pulses into precise register-level actions without relying on microprogramming.
- To use sequential logic and flip-flops to manage clock-based execution cycles, constructing a reliable sequence counter and synchronization system that drives each instruction phase through well-defined timing states (T0–T15).
- To create a modular, readable, and verifiable CPU design using Logisim Evolution, ensuring each component—registers, ALU, memory, control unit, buses—is independently testable yet fully integrated into the complete processor workflow.
- To test the execution of sample machine programs by verifying register and memory changes, ensuring that each instruction is properly interpreted and executed, and that data is correctly stored, modified, and retrieved as expected throughout the instruction cycle.

ASSUMPTIONS AND INSTRUMENTS REQUIRED

ASSUMPTIONS:

1. Implementation of Interrupt is to be neglected
2. Implementation of Input-Output is neglected

INSTRUMENTS REQUIRED FOR DESIGN OF OUR CPU:

1. A memory unit with 4096 words of 16 bits each.
2. Seven registers: AR, PC, DR, IR, TR, and AC.
3. One flip-flops: E ,S(HALT).
4. 1-decoders: a 3X8 opcode decoder and 1 decoder: a 4X16 timing decoder.
5. A 16-bit common bus.
6. ALU and Common Bus Encoder.
7. ALU connected to the input of AC.
8. A 4-bit sequence counter.
9. A 12-bit to 16-bit converter and 16-bit to 12-bit converter.

ARCHITECTURE

Understanding the Data Path in a CPU

The data path is a crucial part of a CPU that manages the movement of data and the execution of arithmetic and logical operations. It consists of several interconnected components that work together to carry out the CPU's tasks efficiently. Below is an overview of the key components that make up the data path in a 16-bit CPU design based on Morris Mano's model:

1. Arithmetic Logic Unit (ALU)

The ALU is the heart of the data path, responsible for performing arithmetic and logical operations on 16-bit data. It includes essential sub-components such as adders, comparators, and logic gates. The ALU processes data based on control signals and outputs the result to designated registers.

2. Program Counter (PC)

The PC is a 12-bit register that holds the memory address of the next instruction to be executed. After fetching an instruction, the PC is automatically incremented to point to the following instruction in memory.

3. Instruction Register (IR)

The IR is a 16-bit register that temporarily stores the current instruction fetched from memory. The control unit decodes this instruction, generating control signals that govern the operation of the data path components.

4. Accumulator (AC)

The AC is a 16-bit register that stores the intermediate and final results of arithmetic and logical operations performed by the ALU. It is often involved in load, store, and arithmetic operations.

5. Data Register (DR)

The DR is a 16-bit register used to hold data fetched from memory or input devices. This data is then transferred to the ALU or other registers for further processing.

6. Address Register (AR)

The AR holds memory addresses involved in data operations — either the address from which data will be fetched to a register or the address where data will be stored. It plays a key role in memory operations.

7. Temporary Register (TR)

The TR is a 16-bit register used for holding intermediate data during instruction execution. It is especially useful when an operation requires multiple steps or temporary storage before transferring data to the final destination.

These components work together to enable the CPU to perform its tasks. When an instruction is

fetched from memory, it is loaded into the IR. The PC is then incremented to point to the next instruction. The instruction is then decoded, and the appropriate control signals are generated to direct the operation of the data path.

The use of the IR and TR in the data path can help to simplify the execution of complex instructions and improve the performance of the CPU by reducing the number of memory accesses required during instruction execution.

Sequence counter:

A sequence counter in a CPU is a digital circuit element that is used to generate a sequence of values based on a clock signal. The sequence counter is typically used to control the flow of instructions in the CPU.

In a typical CPU design, the sequence counter is connected to the program counter (PC) register and the memory unit. The sequence counter generates a sequence of memory addresses that are used to fetch the instructions from the memory unit and load them into the instruction register (IR) of the CPU. The PC register then stores the address of the next instruction to be executed.

When an instruction is executed, the sequence counter increments the PC register to point to the next instruction in memory. This process continues until the program is complete.

The sequence counter is typically implemented as a binary counter that counts from 0 to a maximum value, and then wraps around to 0 again. The counter is incremented on every clock cycle, and the output of the counter is used as the memory address to fetch the instruction from memory.

In addition to controlling the flow of instructions, the sequence counter can also be used for other purposes in the CPU. For example, it can be used to generate timing signals for other components of the CPU or to implement a loop or a branch in the program execution. Overall, the sequence counter is an important component of the CPU design that is used to control the flow of instructions and ensure the proper execution of the program.

Decoder in Control Unit:

In a CPU, a decoder in the control unit is a digital circuit element that is used to decode instruction opcodes and generate control signals for the different components of the CPU data path.

The control unit is responsible for generating control signals that coordinate the operations of the different components of the CPU data path, such as the arithmetic and logic unit (ALU), registers, and memory unit. The decoder in the control unit is a key component that helps to generate these control signals by decoding the instruction opcodes and determining the appropriate control signals for each instruction.

The decoder typically takes the opcode of the instruction from the instruction register (IR) and decodes it into a set of control signals that are sent to the different components of the CPU data path. For example, the decoder might generate control signals to specify the source and destination registers for an instruction, the type of operation to be performed by the ALU, and the memory addresses for load and store instructions.

The decoder in the control unit can be implemented using a combinational logic circuit that maps each

opcode to a set of control signals, or it can be implemented using a microcode that stores the control signals for each instruction in a lookup table.

Overall, the decoder in the control unit is an important component of the CPU design that helps to generate control signals for the different components of the data path and ensure the proper execution of the program.

Register Reference Instructions:

Register reference instructions are a type of computer instruction that operates on data stored in registers. In these instructions, the operands (data values to be operated on) are specified as the contents of specific registers, rather than as values stored in memory locations.

Register reference instructions offer several advantages over memory reference instructions, where operands are specified by their memory locations. First, register access is much faster than memory access, as registers are located directly on the CPU and can be accessed with fewer clock cycles. Second, using registers can reduce the number of memory accesses required to execute a program, improving overall performance.

Register-reference:

	$D_7I'T_3 = r$ (common to all register-reference instructions)	
	$IR(i) = B_i (i = 0, 1, 2, \dots, 11)$	
	$r :$	$SC \leftarrow 0$
CLA	$rB_{11} :$	$AC \leftarrow 0$
CLE	$rB_{10} :$	$E \leftarrow 0$
CMA	$rB_9 :$	$AC \leftarrow \overline{AC}$
CME	$rB_8 :$	$E \leftarrow \overline{E}$
CIR	$rB_7 :$	$AC \leftarrow \text{shr } AC, \quad AC(15) \leftarrow E, \quad E \leftarrow AC(0)$
CIL	$rB_6 :$	$AC \leftarrow \text{shl } AC, \quad AC(0) \leftarrow E, \quad E \leftarrow AC(15)$
INC	$rB_5 :$	$AC \leftarrow AC + 1$
SPA	$rB_4 :$	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	$rB_3 :$	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	$rB_2 :$	If $(AC = 0)$ then $PC \leftarrow PC + 1$
SZE	$rB_1 :$	If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	$rB_0 :$	$S \leftarrow 0$

Memory Reference Instructions:

Memory reference instructions are those commands or instructions which are in the custom to

Memory-reference:

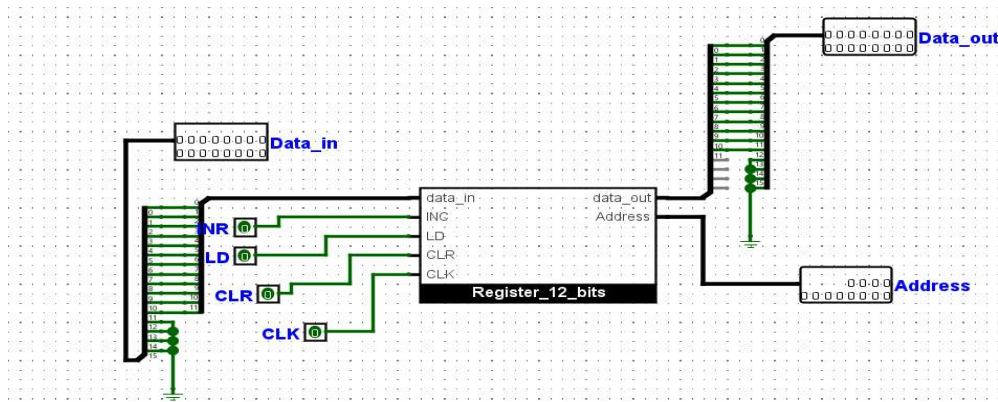
AND	$D_0T_4 :$	$DR \leftarrow M[AR]$
	$D_0T_5 :$	$AC \leftarrow AC \wedge DR, \quad SC \rightarrow 0$
ADD	$D_1T_4 :$	$DR \leftarrow M[AR]$
	$D_1T_5 :$	$AC \leftarrow AC + DR, \quad E \leftarrow C_{out} \rightarrow SC \leftarrow 0$
LDA	$D_2T_4 :$	$DR \leftarrow M[AR]$
	$D_2T_5 :$	$AC \leftarrow DR, \quad SC \leftarrow 0$
STA	$D_3T_4 :$	$M[AR] \leftarrow AC, \quad SC \leftarrow 0$
BUN	$D_4T_4 :$	$PC \leftarrow AR, \quad SC \leftarrow 0$
BSA	$D_5T_4 :$	$M[AR] \leftarrow PC, \quad AR \leftarrow AR + 1$
	$D_5T_5 :$	$PC \leftarrow AR, \quad SC \leftarrow 0$
ISZ	$D_6T_4 :$	$DR \leftarrow M[AR]$
	$D_6T_5 :$	$DR \leftarrow DR + 1$
	$D_6T_6 :$	$M[AR] \leftarrow DR, \quad \text{if } (DR = 0) \text{ then}$ $(PC \leftarrow PC + 1), \quad SC \leftarrow 0$

generate a reference to the memory and approval to a program to have an approach to the commanded information and that states as to from where the data is cache conta

CIRCUIT SCREENSHOTS AND EXPLANATION

1. REGISTERS

ADDRESS REGISTER

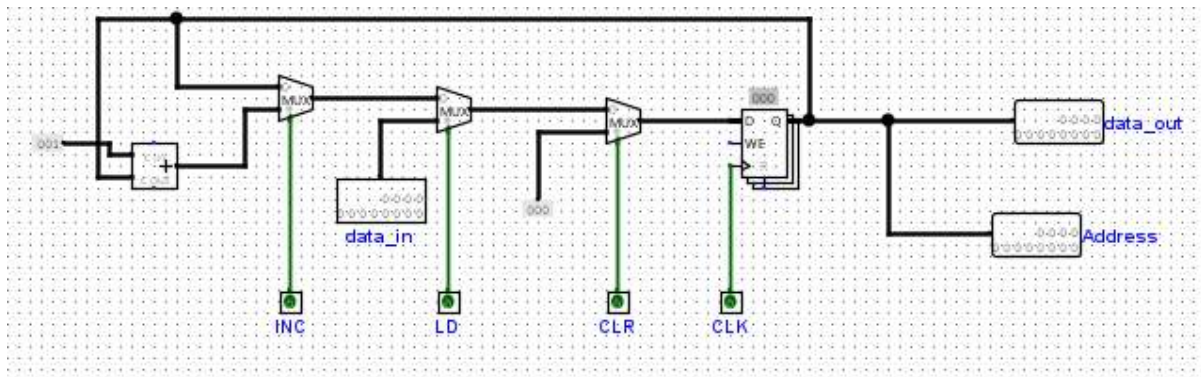


Function of the Address Register

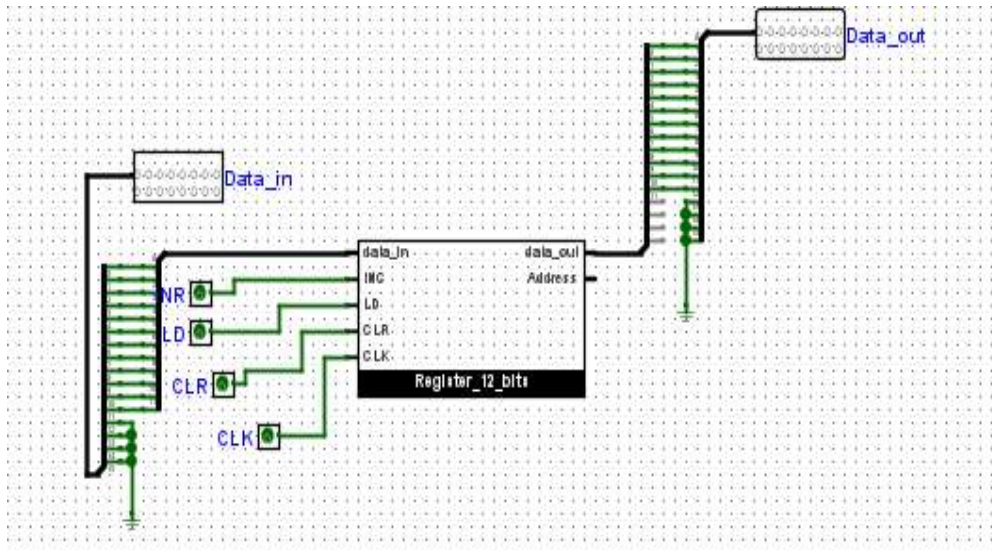
The Address Register is a special-purpose register used to store the address of memory locations during CPU execution. This address is then used for read/write operations with RAM or ROM. It is often also referred to as the Memory Address Register (MAR).

- logic: CLR (Clear): Resets the address register to 0000h.
- LD (Load Enable): Allows new data from Data_in to be loaded.
- INR (Increment): to increase the value of AR to move to next address.
- CLK (Clock): Synchronizes all operations (rising edge trigger).
-

GENERAL 12 BIT REGISTER

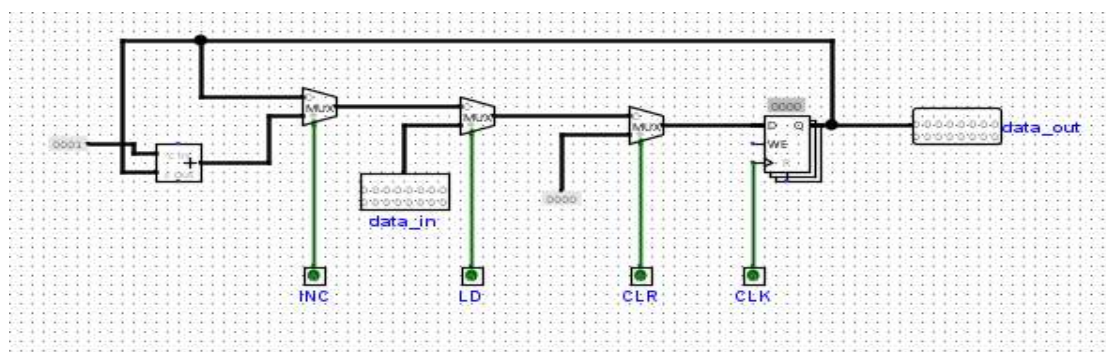


PROGRAM COUNTER



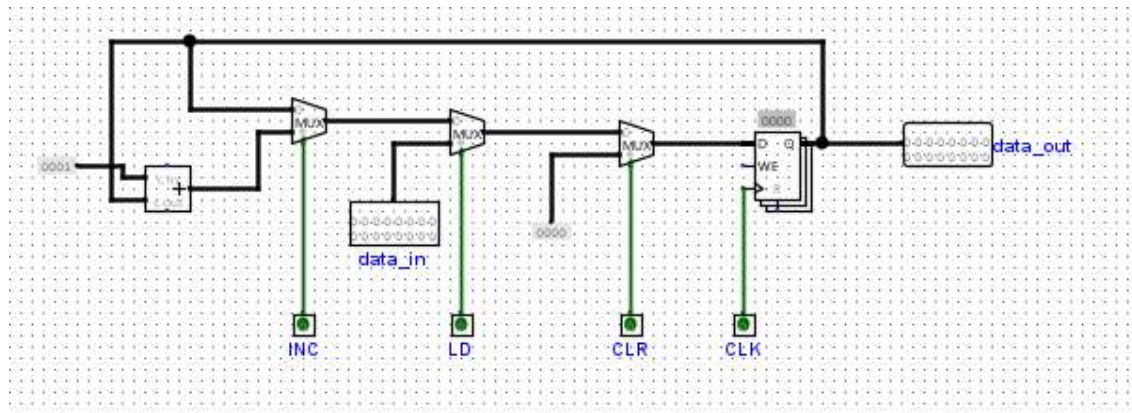
- Also known as:
 - **Instruction Pointer** (in Intel x86 and Itanium architectures),
 - **Instruction Address Register**,
 - **Instruction Counter**,
 - Or part of the **Instruction Sequencer**.
- Holds the **address of the next instruction** to be executed in the program sequence.
- Keeps the **CPU on track** during program execution by progressing through instructions step by step.
- This is also a **12-bit register**, as it holds memory addresses just like the AR.
- Shares the **same hardware circuitry** as the AR register.

DATA REGISTER



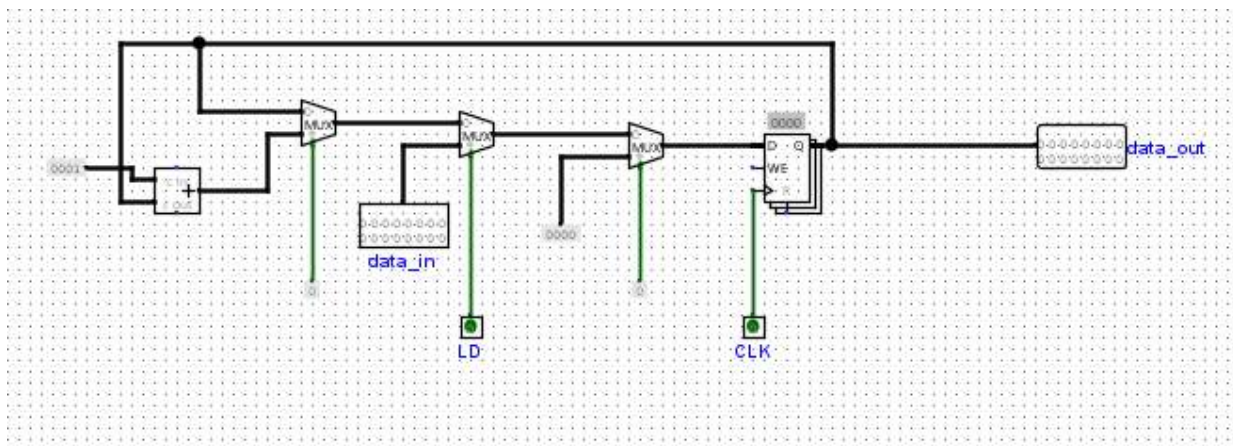
- Holds the **operand/data** for processing.
- Connected to the **ALU**, enabling direct participation in operations.
- Also referred to as the **Memory Buffer Register (MBR)** or **Memory Data Register (MDR)**.
- Handles data **being transferred to/from immediate access storage**.
- This is a **16-bit register**.

ACCUMULATOR



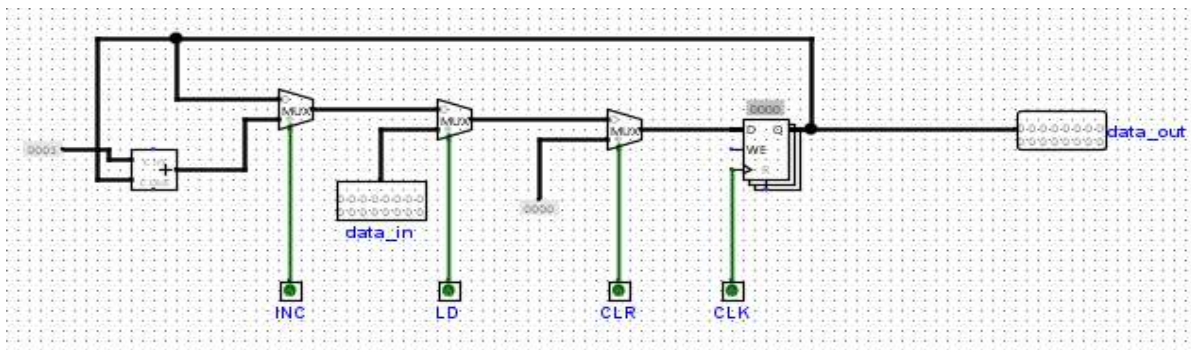
- Stores the **results of arithmetic operations** performed by the ALU.
- Connected to the **common bus through the ALU**, not directly to the bus.
- Acts as the **intermediate storage** for arithmetic/logic results.
- This is a **16-bit register**, as the CPU supports 16-bit data size.

INSTRUCTION REGISTER



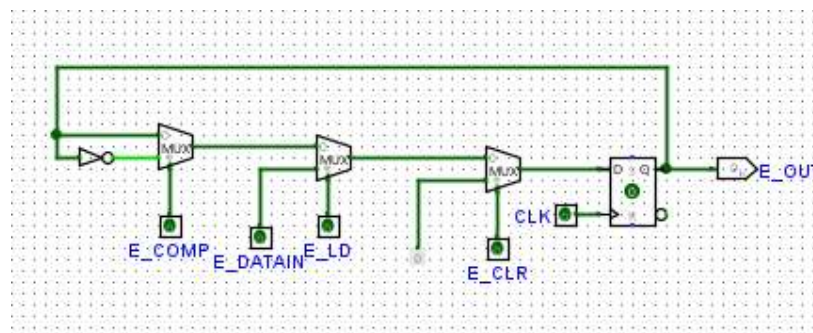
- Temporarily stores the **current instruction** fetched from memory.
- Connected to the **Control and Timing Unit**, where:
 - The **4 MSBs (bits 15-12)** are decoded:
 - **Bit 15 (MSB) = Mode Bit ('I')**
 - **Bits 14-12 = Opcode (Instruction bits)**
 - **Bits 11-0 = Address bits**
- This is a **16-bit register**.

TEMPORARY REGISTER



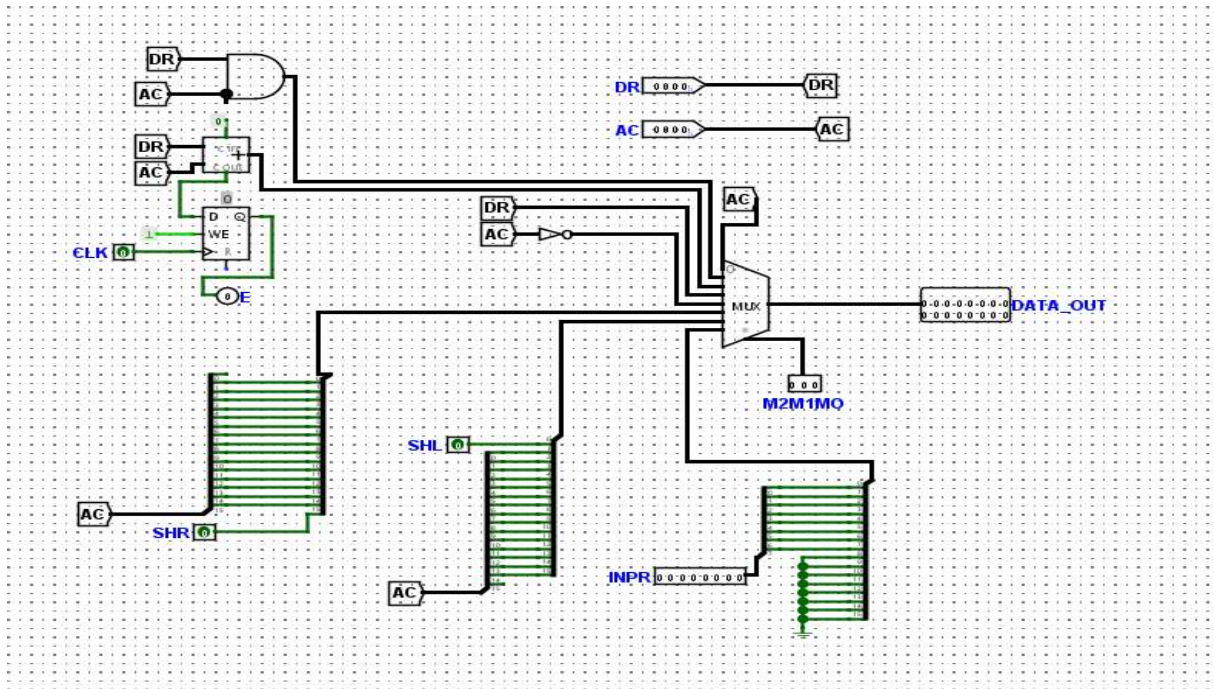
- Used to **temporarily store data**, such as the next program instruction during certain operations (e.g., interrupts).
- Can be **read and written multiple times** in a single instruction:
 - **Single-write and triple-read access.**
 - Supports instructions with **up to three temporary registers** as inputs.
- Not used in this project, as **interrupts are not considered**.
- This is a **16-bit register**.

E FIIP FLOP



- The E Register is used for storing and handling the carry bit generated during addition operations in the ALU. This register is essential for managing carry-over values in multi-bit arithmetic operations, especially in 16-bit addition.
- It has control units complement E, clear and load

2) ARITHMETIC AND LOGIC UNIT



ALU Design Overview (16-bit CPU, Morris Mano Architecture)

1. Core Role:

The Arithmetic Logic Unit (ALU) is a key component responsible for performing arithmetic and logical operations on data within the CPU.

2. Inputs:

- 16-bit input from the Accumulator (AC)
- 16-bit input from the Data Register (DR)

These inputs are processed based on the instruction being executed.

3. Supported Operations:

The ALU can perform the following operations:

- **AND:** Bitwise AND between AC and DR.
- **ADD:** Adds the contents of AC and DR.
- **CMA:** Complements (inverts) the AC contents.
- **INC:** Increments the AC by one.
- **CLE:** Clears the carry bit (E flip-flop).
- **CLA:** Clears the Accumulator (AC).
- **CIR:** Circular right shift through the carry bit.
- **CIL:** Circular left shift through the carry bit.

4. Carry Bit (E Flip-Flop):

- Used for handling **carry/extend bits** in addition and circular shifts.
- Helps maintain accuracy in operations involving overflows or bit rotations.

5. Bit Manipulation:

- Supports **shift left and shift right** operations on AC contents.
- Includes **circular shifts** using the E flip-flop for efficient bit rotation.

6. **Internal Structure:**

- Built using **AND gates, adders, inverters, shifters**, and other logic components.
- Outputs from these are fed into a **multiplexer**.

7. **Output Selection:**

- A **3-bit control signal** from the Control Unit selects the final ALU output using the multiplexer.
- The selected result is placed onto the **data bus** and can be loaded back into AC or other registers.

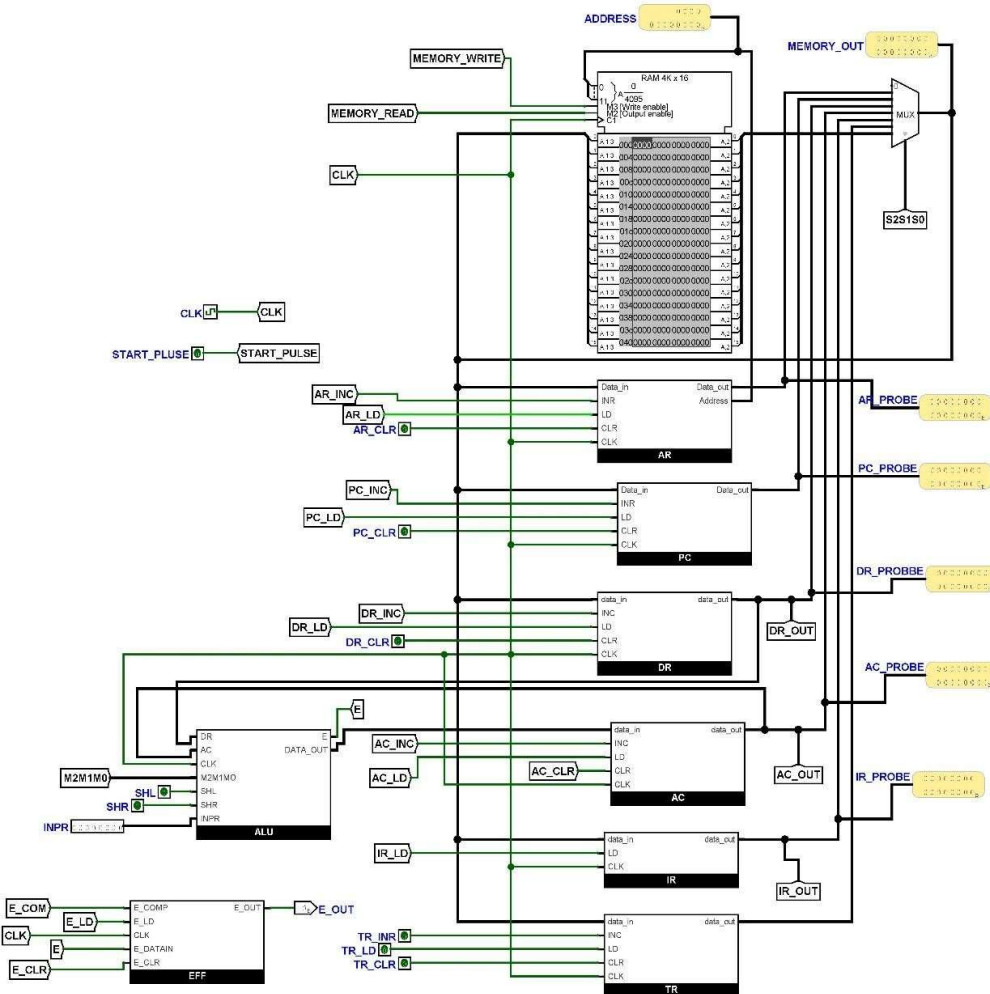
8. **Design Alignment:**

- Follows the **task and opcode structure** of Morris Mano's model.
- Maintains a **clear and modular structure** for easy simulation in **Logisim Evolution**.

9. **Importance:**

- Enables the CPU to effectively execute its instruction set.
- Plays a vital role in **arithmetic, logical, and data transfer operations** within the instruction cycle

3) COMMON BUS



1. Unified Data Path:

- A set of parallel lines (e.g. 16 lines for a 16-bit system) transfers data between CPU components.

2. Connected Components:

- Registers (AR, DR, AC, IR, TR, PC)
- ALU
- Memory interface
- All connected to the bus via tri-state buffers, multiplexers, and load-enable circuits.

3. Data Transfer Control:

- Only one component can place data on the bus at a time using output enable signals.
- Multiple components can receive data from the bus, controlled by load control signals.

Control Mechanism:

1. **Control Unit Function:**
 - Generates **timing and control signals** to manage all data transfers and operations.
2. **Output Control Lines:**
 - Decide which register or unit sends data to the bus.
3. **Input Control Lines:**
 - Decide which register or component loads (captures) data from the bus.
4. **ALU Operation Control:**
 - Managed by control signals, receiving data via either **internal buses** or **direct connections** (e.g. from AC and TR).

Advantages of Common Bus Architecture:

1. **Hardware Efficiency:**
 - Minimizes the number of data lines and connections.
2. **Centralized Data Flow:**
 - Simplifies routing and overall control logic.
3. **Scalability:**
 - Makes it easier to add new components or registers.
4. **Modularity:**
 - Supports a clear, structured, and organized CPU design.

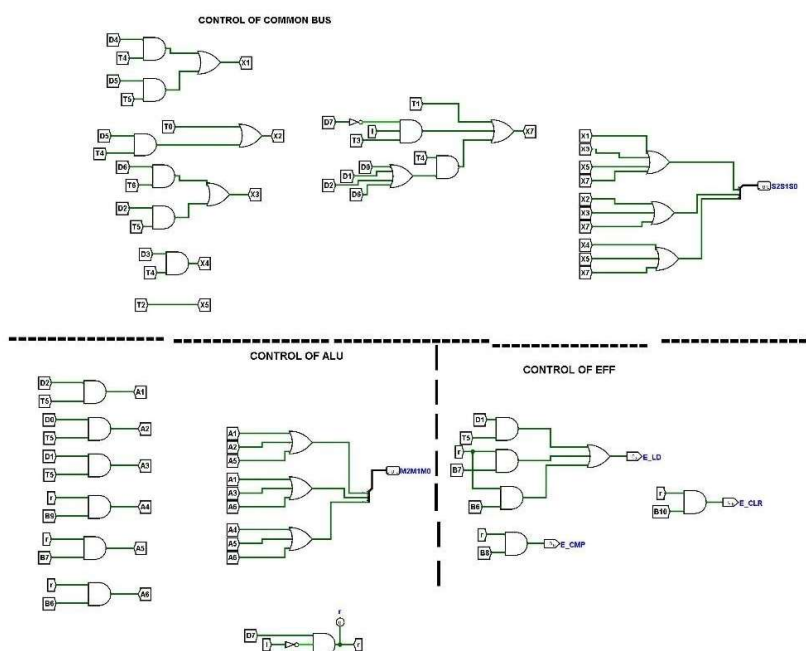
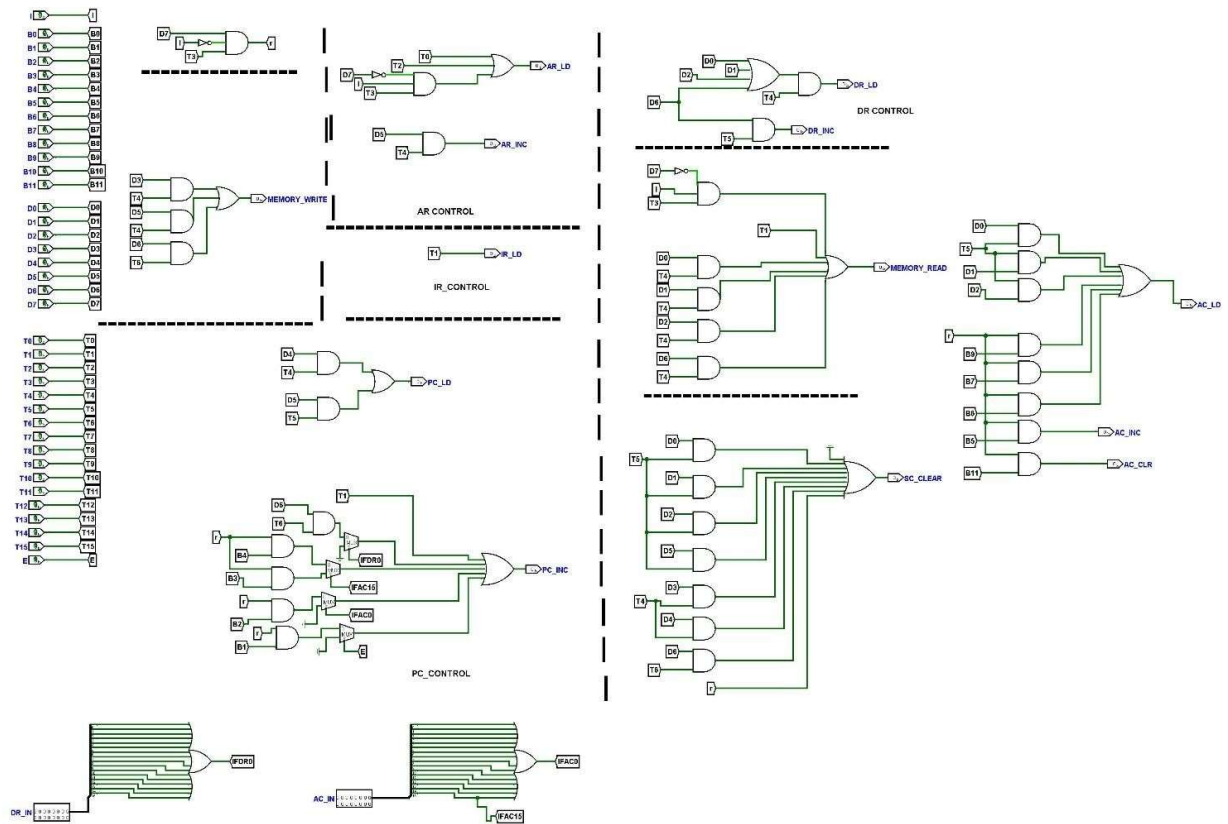
4) Controls

TABLE 5-6 Control Functions and Microoperations for the Basic Computer

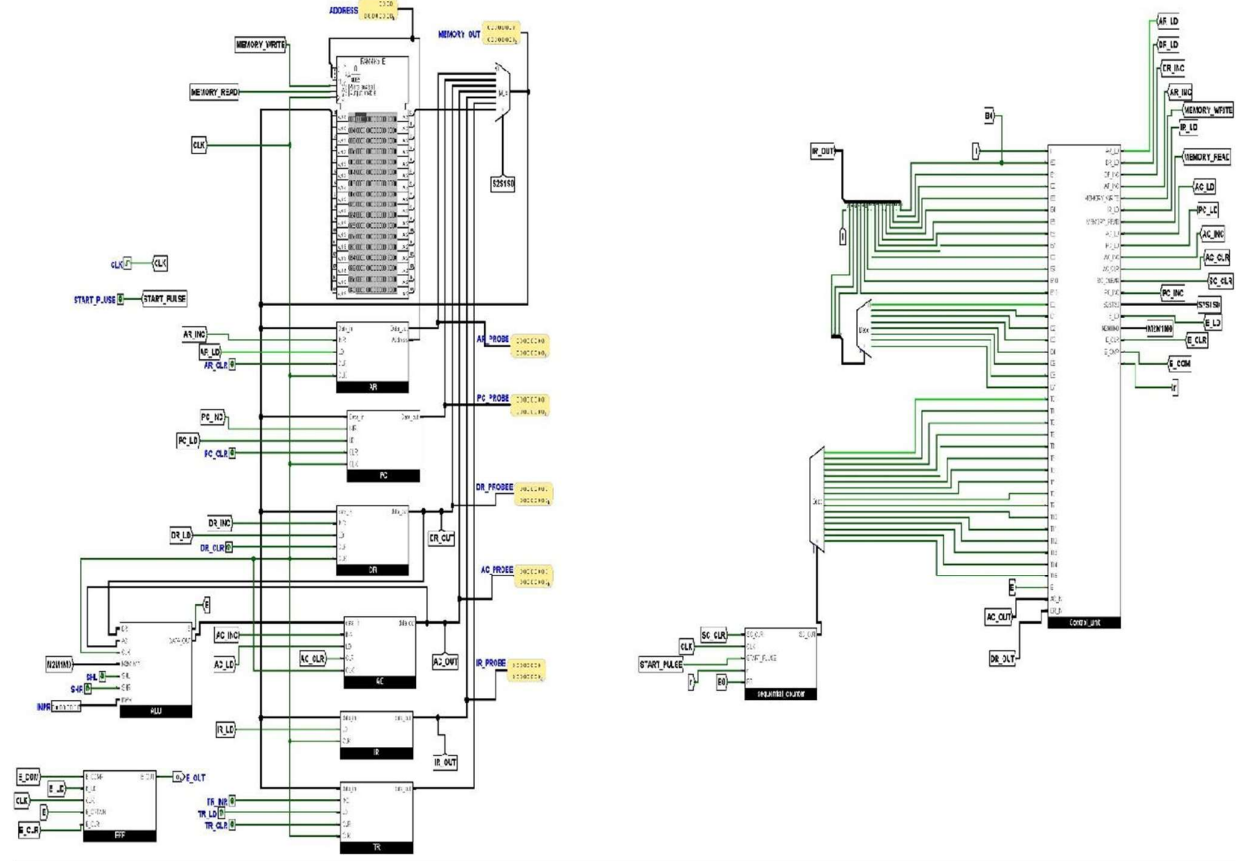
Fetch	$R'T_0$:	$AR \leftarrow PC$
	$R'T_1$:	$IR \leftarrow M[AR], PC \leftarrow PC + 1$
Decode	$R'T_2$:	$D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14),$ $AR \leftarrow IR(0-11), I \leftarrow IR(15)$
Indirect	D_7IT_3 :	$AR \leftarrow M[AR]$
Interrupt:		
	$T_0T_1T_2(IEN)(FGI + FGO)$:	$R \leftarrow 1$
	RT_0 :	$AR \leftarrow 0, TR \leftarrow PC$
	RT_1 :	$M[AR] \leftarrow TR, PC \leftarrow 0$
	RT_2 :	$PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$
Memory-reference:		
AND	D_0T_4 :	$DR \leftarrow M[AR]$
	D_0T_5 :	$AC \leftarrow AC \wedge DR, SC \leftarrow 0$
ADD	D_1T_4 :	$DR \leftarrow M[AR]$
	D_1T_5 :	$AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$
LDA	D_2T_4 :	$DR \leftarrow M[AR]$
	D_2T_5 :	$AC \leftarrow DR, SC \leftarrow 0$
STA	D_3T_4 :	$M[AR] \leftarrow AC, SC \leftarrow 0$
BUN	D_4T_4 :	$PC \leftarrow AR, SC \leftarrow 0$
BSA	D_5T_4 :	$M[AR] \leftarrow PC, AR \leftarrow AR + 1$
	D_5T_5 :	$PC \leftarrow AR, SC \leftarrow 0$
ISZ	D_6T_4 :	$DR \leftarrow M[AR]$
	D_6T_5 :	$DR \leftarrow DR + 1$
	D_6T_6 :	$M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$
Register-reference:		
	$D_7I'T_3 = r$	(common to all register-reference instructions)
	$IR(i) = B_i$	($i = 0, 1, 2, \dots, 11$)
	r :	$SC \leftarrow 0$
CLA	rB_{11} :	$AC \leftarrow 0$
CLE	rB_{10} :	$E \leftarrow 0$
CMA	rB_9 :	$AC \leftarrow \overline{AC}$
CME	rB_8 :	$E \leftarrow \overline{E}$
CIR	rB_7 :	$AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
CIL	rB_6 :	$AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
INC	rB_5 :	$AC \leftarrow AC + 1$
SPA	rB_4 :	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	rB_3 :	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	rB_2 :	If $(AC = 0)$ then $PC \leftarrow PC + 1$
SZE	rB_1 :	If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	rB_0 :	$S \leftarrow 0$
Input-output:		
	$D_7IT_3 = p$	(common to all input-output instructions)
	$IR(i) = B_i$	($i = 6, 7, 8, 9, 10, 11$)
	p :	$SC \leftarrow 0$
INP	pB_{11} :	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$
OUT	pB_{10} :	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$
SKI	pB_9 :	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$
SKO	pB_8 :	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$
ION	pB_7 :	$IEN \leftarrow 1$
IOF	pB_6 :	$IEN \leftarrow 0$

From: Computer System Architecture-morris mano Third Edition

Symbol	Hexadecimal code		Description
	<i>I</i> = 0	<i>I</i> = 1	
AND	0xxx	8xxx	AND memory word to <i>AC</i>
ADD	1xxx	9xxx	Add memory word to <i>AC</i>
LDA	2xxx	Axxx	Load memory word to <i>AC</i>
STA	3xxx	Bxxx	Store content of <i>AC</i> in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA		7800	Clear <i>AC</i>
CLE		7400	Clear <i>E</i>
CMA		7200	Complement <i>AC</i>
CME		7100	Complement <i>E</i>
CIR		7080	Circulate right <i>AC</i> and <i>E</i>
CIL		7040	Circulate left <i>AC</i> and <i>E</i>
INC		7020	Increment <i>AC</i>
SPA		7010	Skip next instruction if <i>AC</i> positive
SNA		7008	Skip next instruction if <i>AC</i> negative
SZA		7004	Skip next instruction if <i>AC</i> zero
SZE		7002	Skip next instruction if <i>E</i> is 0
HLT		7001	Halt computer
INP		F800	Input character to <i>AC</i>
OUT		F400	Output character from <i>AC</i>
SKI		F200	Skip on input flag
SKO		F100	Skip on output flag
ION		F080	Interrupt on
IOF		F040	Interrupt off



Complete Circuit With Integrate



Role and Function:

1. **Manages and coordinates all CPU activities** by generating appropriate control signals.
2. **Interprets the instruction stored in the Instruction Register (IR).**
3. **Orchestrates the sequence of micro-operations** required to execute each instruction.

Hardwired Control Approach:

1. **Fixed Logic Circuits:**
 - Control signals are produced using **combinational logic circuits**.
2. **Based on Timing and Opcode:**
 - Operates on the combination of the **current timing pulse (T0, T1, T2, ...)** and the **decoded opcode**.

Instruction Format and Decoding:

1. **16-bit Instruction Format:**
 - IR[12–15] → **Opcode (4 bits)**
 - IR[0–11] → **Address (12 bits)**
2. **Instruction Decoder:**
 - Decodes the **opcode** to activate one of the control lines (D0 to D7) for memory-reference instructions.
3. **Timing Signals:**
 - Generated by a **binary sequence counter** synchronized with the **system clock**.

Control Signal Generation:

1. **Control Table Reference:**
 - Uses a predefined table mapping **micro-operations to instructions and timing steps**.
2. **Logical Expressions:**
 - Control signals are derived using **AND, OR, and NOT gates** based on the control table.

Tasks Performed by Control Signals:

- **Enable register transfers** between CPU components.
- **Initiate memory read/write operations.**
- **Trigger arithmetic or logical operations** in the ALU.
- **Control conditional branching and instruction sequencing.**

Advantages of Hardwired CU:

1. **Fast and efficient** due to direct hardware implementation.
2. **Low hardware cost** for fixed instruction sets.
3. **Well-suited for simple, predefined CPUs** like the Morris Mano design.

CONCLUSION

This project focused on the complete design, construction, and simulation of a 16-bit Central Processing Unit (CPU), modeled after the Morris Mano computer architecture, using Logisim Evolution. The CPU was designed entirely from scratch, incorporating essential architectural elements such as registers, a shared bus system, an Arithmetic Logic Unit (ALU), and a hardwired control unit.

The CPU's functionality was built around a set of key internal registers including the Accumulator (AC), Data Register (DR), Address Register (AR), Program Counter (PC), Instruction Register (IR), and Temporary Register (TR). Together, these components enabled the CPU to execute a complete and structured instruction cycle, following a clock-driven, time-step-based control system.

A major highlight of the design was the development of a hardwired control unit based on Morris Mano's micro-instruction table. This unit handled the decoding of instructions and generation of precise control signals for register transfers, memory operations, ALU functions, and instruction sequencing — all without relying on microprogramming.

Through this project, core concepts such as instruction fetch, decode, execution flow, register-based operations, and timing control were successfully implemented and demonstrated. The project serves as a practical representation of fundamental computer architecture concepts and lays a strong groundwork for future enhancements like interrupt handling, pipelining, and input/output device integration.

This simulation not only verifies theoretical CPU operation but also provides valuable hands-on experience in digital logic design, control signal coordination, and sequential system behavior within a simplified, educationally effective framework.

REFERENCES

Mano, M. M. (1993). *Computer System Architecture* (3rd ed.). Prentice Hall.